

Compressione di immagini - Fourier e Wavelet

Università degli Studi di Ferrara

Dipartimento di Ingegneria, Matematica e Informatica

Professore:

Prof. **Giacomo DIMARCO**

Studenti:

Dott. **Enrico FAVALE**

Dott. **Alessandro DI TANNA**



**Università
degli Studi
di Ferrara**

Serie e Trasformata di Fourier

- Espansione di una funzione periodica come somma infinita di seni e coseni.
- Valida per funzioni con periodo 2π .
- Forma generale:

$$f(x) = \frac{a_0}{2} + \sum_{k=1}^{\infty} (a_k \cos(kx) + b_k \sin(kx))$$

- Utilizza i coefficienti:

$$a_k = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \cos(kx) dx, \quad b_k = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \sin(kx) dx$$

- La Serie di Fourier può essere espressa in forma complessa:

$$f(x) = \sum_{k=-\infty}^{\infty} c_k e^{ikx}$$

- Il coefficiente c_n è dato da:

$$c_n = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(x) e^{-ikx} dx$$

- È molto utile per il passaggio alla Trasformata di Fourier.

- Generalizzazione della Serie di Fourier a funzioni non periodiche.
- Trasforma una funzione dal dominio del tempo al dominio della frequenza.
- Formula:

$$\hat{f}(\omega) = \mathcal{F}(f(\omega)) = \int_{-\infty}^{\infty} f(x) e^{-i\omega x} dx$$

- La trasformata inversa consente di recuperare la funzione originale:

$$f(x) = \mathcal{F}^{-1}(\hat{f}(x)) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \hat{f}(\omega) e^{i\omega x} d\omega$$

- Applicabile a segnali discreti e di lunghezza finita.
- La DFT è usata per analisi numeriche e digitali.
- Formula:

$$\hat{f}[k] = \sum_{n=0}^{N-1} f[n] \cdot e^{-i2\pi kn/N}, \quad \text{per } k = 0, 1, \dots, N-1$$

- Richiede $O(N^2)$ operazioni.
- La trasformata inversa (IDFT) permette di tornare nel dominio del tempo:

$$f[n] = \frac{1}{N} \sum_{k=0}^{N-1} \hat{f}[k] \cdot e^{i2\pi kn/N}, \quad \text{per } n = 0, 1, \dots, N-1$$

- Algoritmo efficiente per calcolare la DFT.
- Complessità ridotta a $O(N \log N)$.
- Fondamentale per applicazioni in elaborazione del segnale, immagini, compressione.
- Separa il segnale $f[n]$ di lunghezza N in due sotto-sequenze di lunghezza $N/2$:

$$f_{\text{pari}}[n] = f[n] + f[n + N/2], \quad f_{\text{dispari}}[n] = e^{(\frac{-i2\pi n}{N})} (f[n] - f[n - N/2])$$

- Per i k pari vale:

$$\hat{f}[2k] = \sum_{n=0}^{(N/2)-1} (f[n] + f[n + N/2]) e^{(\frac{-i2\pi kn}{N/2})}$$

- Per i k dispari vale:

$$\hat{f}[2k + 1] = \sum_{n=0}^{(N/2)-1} e^{(\frac{-i2\pi n}{N})} (f[n] - f[n + N/2]) e^{(\frac{-i2\pi kn}{N/2})}$$

Trasformata in dimensione due - FFT2

- Estensione della FFT alle immagini (matrici bidimensionali).
- Applicata riga per riga, poi colonna per colonna.
- Formula:

$$\hat{f}[k, l] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m, n] \cdot e^{-i2\pi\left(\frac{km}{M} + \frac{ln}{N}\right)}$$

- Trasformata inversa:

$$f[m, n] = \frac{1}{MN} \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} \hat{f}[k, l] \cdot e^{i2\pi\left(\frac{km}{M} + \frac{ln}{N}\right)}$$

- Usata per la compressione e filtraggio di immagini.

Compressione di immagini usando Fourier

Classe ImageCompressor

```
1 class ImageCompressor:
2     def __init__(self, input_dir, image_name):
3         self.input_dir = input_dir
4         self.image_name = image_name
5         self.image_path = f"{input_dir}/{image_name}"
6
7     def __compress_fft_channel(...):
8         ...
9
10    def compress_color_image_fft(...):
11        ...
```

Compressione di un canale

```
1 def __compress_fft_channel(  
2     self,  
3     channel,  
4     keep_fraction=0.1,  
5     filter="low-pass",  
6     save_spectrum=False,  
7     channel_name="",  
8     output_dir="spectra_output"  
9 ):
```

Trasformata - Funzioni: fft2 e fftshift

```
1 fft = np.fft.fft2(channel)
2 fft_shifted = np.fft.fftshift(fft)
```

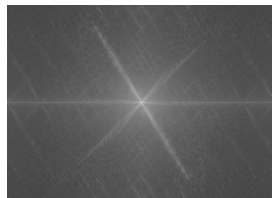
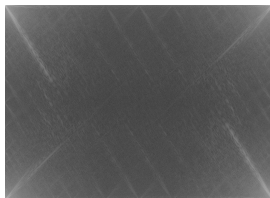


Figura 1: Confronto tra immagini: originale, spettro fft2, spettro fftshift.

Applicazione di una maschera

```
1 rows, cols = channel.shape
2 crow, ccol = rows // 2, cols // 2
3 r_keep = int(rows * keep_fraction / 2)
4 c_keep = int(cols * keep_fraction / 2)
5
6 mask = np.zeros_like(channel, dtype=bool)
7 mask[crow - r_keep : crow + r_keep, ccol - c_keep : ccol + c_keep] = True
8
9 fft_shifted_filtered = fft_shifted.copy()
10 fft_shifted_filtered[~mask] = 0
```

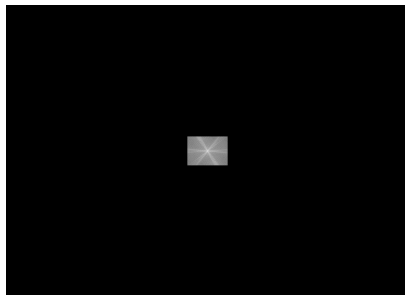
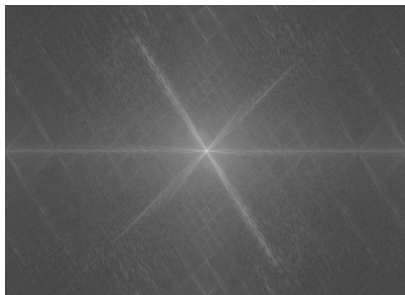


Figura 2: Confronto tra spettro prima (sinistra) e dopo (destra) l'applicazione della maschera.

Trasformata inversa - Funzioni: ifft2 e ifftshift

```
1 fft_inverse = np.fft.ifft2(np.fft.ifftshift(fft_shifted_filtered))  
2 return np.abs(fft_inverse)
```

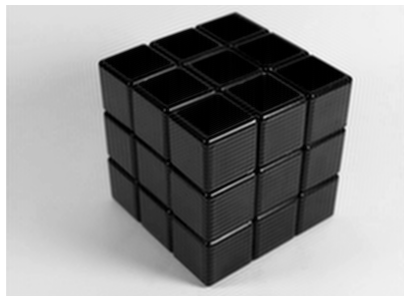
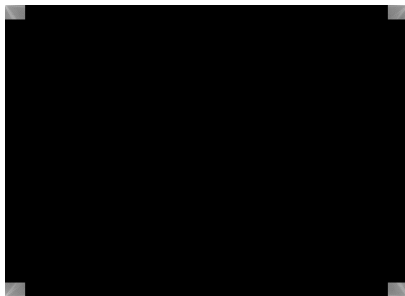


Figura 3: A sinistra: spettro delle frequenze dopo ifftshift. A destra: canale ricostruito dopo ifft2.

Compressione dell'immagine

```
1 def compress_color_image_fft(  
2     self,  
3     keep_fraction=0.1,  
4     filter="low-pass",  
5     show=True,  
6     save_name="compressed.jpg",  
7     save_spectra=True  
8 )
```

Ciclo di compressione dei canali

```
1 img = Image.open(self.image_path).convert("RGB")
2 img_np = np.array(img)
3
4 for i in range(3):
5     channel = img_np[:, :, i]
6     compressed = self.__compress_fft_channel(
7         channel=channel,
8         keep_fraction=keep_fraction,
9         filter=filter,
10        ...
11    )
12     compressed = np.clip(compressed, 0, 255)
13     compressed_channels.append(compressed.astype(np.uint8))
```

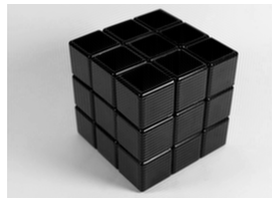


Figura 4: Canali dell'immagine compressi: rosso (R), verde (G) e blu (B), ottenuti dopo la trasformatata inversa.

Ricostruzione dell'immagine compressa

```
1 compressed_img = np.stack(compressed_channels, axis=2)
2 compressed_pil = Image.fromarray(compressed_img)
```

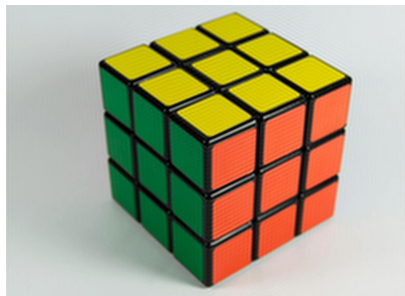
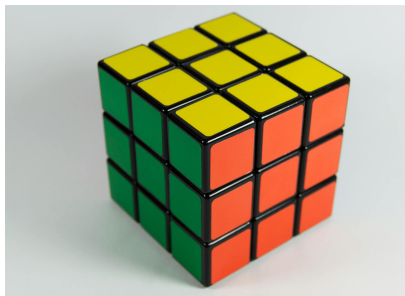


Figura 5: Confronto tra immagine originale (sinistra) e immagine compressa (destra) con `keep_fraction` a 0.1.

Utilizzo di un filtro alternativo

```
1 if filter == "low-pass":  
2     fft_shifted_filtered[~mask] = 0  
3 else:  
4     fft_shifted_filtered[mask] = 0
```

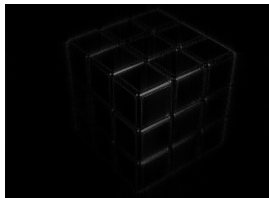
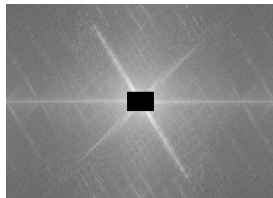


Figura 6: Esempio di compressione dell'immagine utilizzando un filtro “high-pass” invece del classico “low-pass”. Rappresentazione dello spettro delle frequenze già shiftato maschera applicata, del canale Blu (B) compresso e dell'immagine compressa risultante.

Esempi di compressione con entrambi i filtri

Ponte di Brooklyn

Il primo esempio è un'immagine del ponte di Brooklyn di dimensione 2000 x 2000. Viene applicato un filtro “low-pass” e un filtro “high-pass” con `keep_fraction` a 0.1 in entrambi i casi.



Figura 7: Immagine originale del ponte di Brooklyn.

Ponte di Brooklyn - “low-pass” e “high-pass”

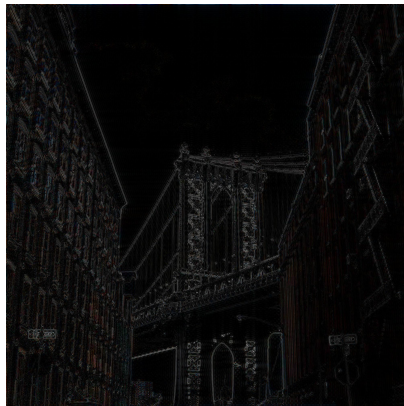


Figura 8: Confronto tra immagine compressa con filtro “low-pass” e immagine compressa con filtro “high-pass”.

Il secondo esempio utilizza l'immagine di un lupo di dimensione 1920 x 1920. Anche in questo caso vengono applicati un filtro “low-pass” e un filtro “high-pass” con `keep_fraction` impostata, questa volta, a 0.05.



Figura 9: Immagine originale di un lupo.

Lupo - “low-pass” e “high-pass”

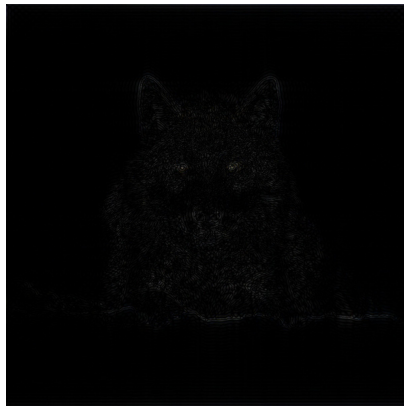


Figura 10: Confronto tra immagine compressa con filtro “low-pass” e immagine compressa con filtro “high-pass”.

Teoria delle Wavelet

- La serie di Fourier non è adatta quando ci si confronta con segnali *non stazionari*.
- Fornisce un'informazione completa sulle **frequenze presenti**, ma non sulla loro **collocazione temporale**.
- Inadeguata per analisi di eventi come picchi, impulsi, discontinuità ecc.

- Versione *Short-Time Fourier Transform*.
- Divide il segnale in finestre temporali, applicando su ognuna la trasformata di Fourier.
- Fornisce una rappresentazione simultanea di **tempo e frequenza**.
- Formula:

$$G[f](t, \omega) = \int_{-\infty}^{\infty} f(\tau) g(\tau - t) e^{-i\omega\tau} d\tau$$

- τ caratterizza la **larghezza** della finestra.
- Limitante avere una dimensione fissa per la finestra.

- Le wavelet consentono alla **dimensione** della finestra di **variare** progressivamente.
- Finestre iniziali più **grandi** catturano meglio **basse frequenze**.
- Man mano che le finestre diventano più **brevi** diventano più adatte a catturare **alte frequenze**.

- Le wavelet utilizzano una **funzione madre** (o **wavelet madre**).
- Formula:

$$\psi_{a,b}(t) = \frac{1}{\sqrt{|a|}} \psi \left(\frac{t-b}{a} \right)$$

- Esistono **diverse** tipologie di wavelet madre, ognuna adatta ad un **compito** diverso.
- Tramite operazioni di *traslazione* e *scalatura* sulla wavelet madre si ottengono altre wavelet adatte a catturare informazioni diverse dal segnale.

- Formula:

$$\psi_{a,b}(t) = \frac{1}{\sqrt{|a|}} \psi\left(\frac{t-b}{a}\right)$$

a e b sono i coefficienti di, rispettivamente, scalatura e traslazione.

- La **scalatura** indica la larghezza della finestra.
- La **traslazione** indica la posizione della finestra nel tempo.

- Il **kernel** utilizzato nella trasformata di Fourier è definito come $K(t, \omega) = e^{-i\omega t}$.
- Nella trasformata wavelet continua la **wavelet madre** funge da kernel.
- Formula:

$$\mathcal{W}_f(a, b) = (f, \psi_{a,b}) = \int_{-\infty}^{\infty} f(t) \bar{\psi}_{a,b}(t) dt$$

Rappresentazione dell'analisi tempo - frequenza

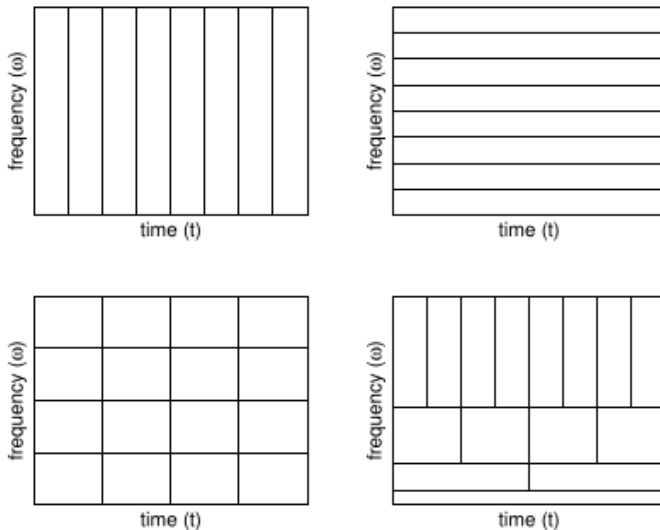


Figura 11: Differenze tra analisi di serie temporale, analisi di Fourier, analisi di Gabòr e analisi wavelet per un segnale.

- La discretizzazione avviene definendo un **lattice**:

$$\psi_{m,n}(x) = a_0^{-m/2} \psi(a_0^{-m}x - nb_0), \quad \text{con } a_0, b_0 > 0 \text{ e } m, n \text{ interi}$$

- La **trasformata wavelet discreta** è definita:

$$\mathcal{W}_\psi(f)(m, n) = \int_{-\infty}^{\infty} f(t) \bar{\psi}_{m,n}(t) dt = a_0^{-m/2} \int_{-\infty}^{\infty} f(t) \bar{\psi}(a_0^{-m}(t) - nb_0) dt$$

- L'ortogonalità risulta fondamentale in quanto implica **indipendenza** tra le wavelet.
- Si definisce condizione di **ortogonalità**:

$$(\psi_{m,n}, \psi_{k,l}) = \int_{-\infty}^{\infty} \psi_{m,n}(x) \psi_{k,l}(x) dx = \delta_{m,k} \delta_{n,l}$$

- dove δ è la *delta di Dirac*:

$$\delta_{ij} = \begin{cases} 1 & \text{se } i = j, \\ 0 & \text{se } i \neq j. \end{cases}$$

- Una wavelet è **ortogonale** se la famiglia di funzioni $\psi_{m,n}$ è ortogonale.
- Ne consegue che un segnale può essere espresso **univocamente**:

$$f(t) = \sum_{m,n} c_{m,n} \psi_{m,n}(t)$$

dove i coefficienti $c_{m,n}$:

$$c_{m,n} = \langle f, \psi_{m,n} \rangle$$

- Il potere delle wavelet sta nella capacità di esprimere un segnale con *approssimazioni* via via più fini nel tempo.
- Queste approssimazioni corrispondono a diversi livelli di **risoluzione**.
- L'**analisi multi-risoluzione (MRA)** fornisce un approccio formale per costruire un segnale con diversi livelli di risoluzione.

Compressione di immagini usando Wavelet

Compressione di un canale

```
1 def __compress_wavelet_channel(  
2     self,  
3     channel,  
4     wavelet="haar",  
5     level=1,  
6     threshold=20,  
7     keep_ll_only=True,  
8     save_coeffs=False,  
9     channel_name="",  
10    output_dir="wavelet_coeffs"  
11 )
```

Trasformata - Funzione: wavedec2

```
1 coeffs = pywt.wavedec2(channel, wavelet=wavelet, level=level)
2 coeffs_compressed = []
```

- wavedec2 applica la **trasformata wavelet discreta 2D**.
- Decompone il canale in **sottobande di frequenza** su più **livelli**.
- Ogni livello produce:
 - una componente di **approssimazione** (basse frequenze),
 - tre componenti di **dettaglio** (orizzontale, verticale, diagonale).
- Il parametro wavelet specifica la **funzione wavelet** usata (es. "haar", "db2", ecc.).
- Il parametro level controlla la **profondità della decomposizione**.
- coeffs contiene tutti i **coefficienti** ottenuti, organizzati per livello.

Livelli, Threshold, Matrici da mantenere

```
1 for i, coeff in enumerate(coeffs):
2     if i == 0:
3         LL = pywt.threshold(coeff, threshold, mode='soft')
4         coeffs_compressed.append(LL)
5     else:
6         LH, HL, HH = coeff
7         if keep_ll_only:
8             coeffs_compressed.append((
9                 np.zeros_like(LH),
10                np.zeros_like(HL),
11                np.zeros_like(HH)
12            ))
13        else:
14            LH = pywt.threshold(LH, threshold, mode='soft')
15            HL = pywt.threshold(HL, threshold, mode='soft')
16            HH = pywt.threshold(HH, threshold, mode='soft')
17            coeffs_compressed.append((LH, HL, HH))
```

```
1 reconstructed = pywt.waverec2(coeffs_compressed, wavelet=wavelet)
2 return np.clip(reconstructed, 0, 255)
```

- waverec2 esegue la **ricostruzione del canale** a partire dai **coefficienti wavelet**.
- Usa la stessa **wavelet** della decomposizione per garantire coerenza.
- La funzione restituisce una **versione approssimata** del canale originale.
- `np.clip` garantisce che i valori del canale siano nel range `[0, 255]`.

Compressione dell'immagine

```
1 def compress_color_image_dwt(  
2     self,  
3     wavelet="haar",  
4     level=1,  
5     threshold=20,  
6     keep_ll_only=True,  
7     ...  
8 )
```

Ciclo di compressione dei canali e ricostruzione dell'immagine compressa

```
1 compressed_channels = []
2 channel_names = ['Rosso', 'Verde', 'Blu']
3
4 for i in range(3):
5     channel = img_np[:, :, i]
6     compressed = self.__compress_wavelet_channel(
7         channel=channel,
8         wavelet=wavelet,
9         threshold=threshold,
10        keep_ll_only=keep_ll_only,
11        ...
12    )
13    compressed_channels.append(compressed.astype(np.uint8))
14
15 compressed_img = np.stack(compressed_channels, axis=2)
16 compressed_pil = Image.fromarray(compressed_img)
```

Esempio di compressione del cubo di Rubik con Wavelet



Figura 12: Immagine originale del cubo di Rubik.

Compressione leggera

```
1 rubik.compress_color_image_dwt(  
2     wavelet="db2",  
3     level=2,  
4     threshold=50,  
5     keep_ll_only=False,  
6     show=True,  
7     save_name="compressed_light.jpg"  
8     ),  
9     save_coeffs=True
```

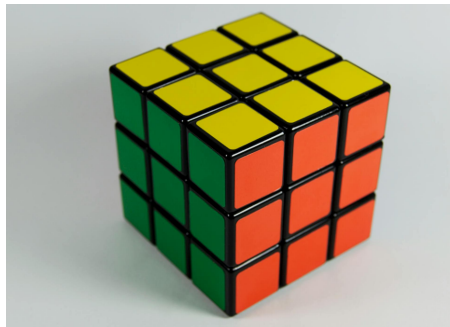


Figura 13: Immagine compressa in maniera leggera.

Compressione media

```
1 rubik.compress_color_image_dwt(  
2     wavelet="haar",  
3     level=3,  
4     threshold=80,  
5     keep_ll_only=False,  
6     show=True,  
7     save_name="compressed_medium.  
8         jpg",  
9     save_coeffs=True  
10 )
```



Figura 14: Immagine compressa in maniera media.

Compressione LL_only

```
1 rubik.compress_color_image_dwt(  
2     wavelet="db2",  
3     level=3,  
4     threshold=100,  
5     keep_ll_only=True,  
6     show=True,  
7     save_name="compressed_LLonly.  
8         jpg",  
9     save_coeffs=True  
10 )
```



Figura 15: Immagine compressa in maniera LL_only.

Compressione pesante

```
1 rubik.compress_color_image_dwt(  
2     wavelet="haar",  
3     level=4,  
4     threshold=150,  
5     keep_ll_only=True,  
6     show=True,  
7     save_name="  
        compressed_aggressive.jpg",  
8     save_coeffs=True  
9 )
```

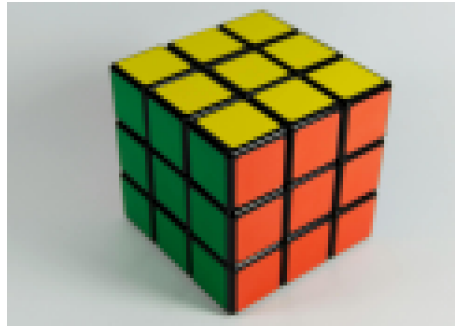


Figura 16: Immagine compressa in maniera pesante.